

Kafka

Monday, 17 August 2026

10:20 pm

Partition

Partitioning takes the **single topic log** and breaks it into **multiple logs**, each of which can live on a separate node in the Kafka cluster. This way, the work of storing messages, writing new messages, and processing existing messages can be split among many nodes in the cluster.

Example:

Imagine you are building a system for logging events in an e-commerce application. You decide to use Kafka to handle these events efficiently. You create a Kafka topic called **ecommerce_events** to store all the events related to user activity on your platform.

Now, let's say your e-commerce application generates various types of events, such as:

1. **User Registration**
2. **Product Viewed**
3. **Add to Cart**
4. **Purchase Completed**

You want to make sure that events of the same type are ordered and processed together, but you also want to handle a high volume of events in parallel. This is where partitions come into play.

You decide to use four partitions for your **ecommerce_events** topic:

- Partition 0: User Registration events
- Partition 1: Product Viewed events
- Partition 2: Add to Cart events
- Partition 3: Purchase Completed events

Here's how the system works:

1. **User Registration Event:** When a user registers, your system produces a "User Registration" event. This event is sent to Partition 0 of the **ecommerce_events** topic.
2. **Product Viewed Event:** When a user views a product, a "Product Viewed" event is produced and sent to Partition 1.
3. **Add to Cart Event:** An "Add to Cart" event goes to Partition 2.
4. **Purchase Completed Event:** The "Purchase Completed" event is sent to Partition 3.

By partitioning the topic in this way, you achieve parallelism because different types of events can be processed independently. For example, you can have consumers dedicated to processing user registrations, others for product views, and so on.

Additionally, within each partition, the order of events is maintained. So, if multiple users register (in Partition 0) or view products (in Partition 1) simultaneously, their events are processed in the order they were received within that partition. This partitioning strategy allows for efficient parallel processing, ordered handling of events within partitions, and the ability to scale the system horizontally by adding more Kafka brokers and partitions.

A great way of visualizing the kafka